

Question 27: Debugging String Matching Code (Skipped Last Valid Position)

Aim

To identify and fix the bug in the brute-force string matching algorithm, ensuring that every valid starting position of the pattern is checked correctly.

Problem Statement

The given brute-force string matching program fails to examine the **last valid starting position** of the pattern within the text because of an incorrect loop boundary.

Given Code:

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
    matches = []  
    for i in range(n - m):  
        for j in range(m):  
            if text[i + j] != pattern[j]:  
                break  
  
        if j == m - 1:  
            matches.append(i)  
  
    return matches
```

Source Code (Corrected)

Brute Force String Matching Algorithm

```
def string_match(text, pattern):  
    # Length of text and pattern  
    n = len(text)
```

```
m = len(pattern)

# List to store all matching positions
matches = []

# Check every valid starting position
for i in range(n - m + 1):

    # Compare pattern with text
    for j in range(m):

        # Stop if characters do not match
        if text[i + j] != pattern[j]:
            break

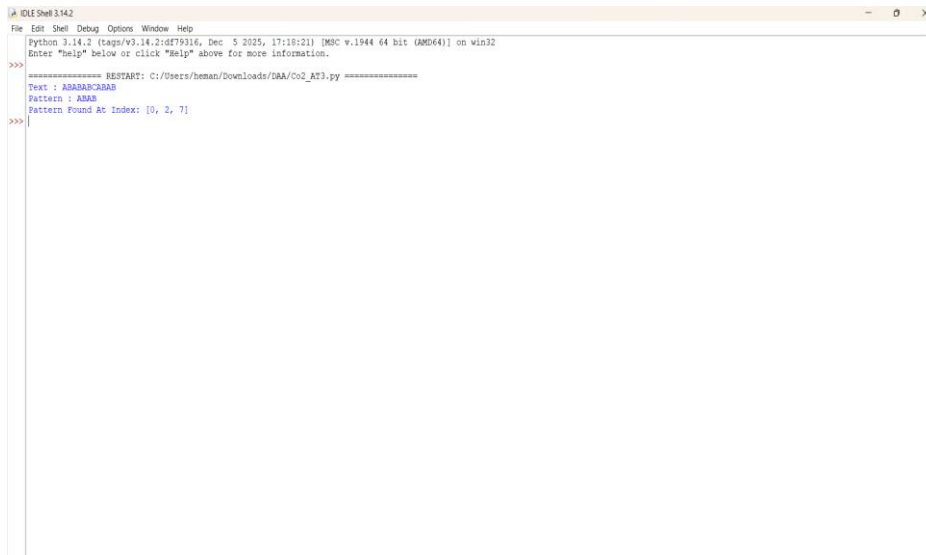
    # If all characters matched
    else:
        matches.append(i)

return matches

# Sample Input
text = "ABABABCABAB"
pattern = "ABAB"

# Display Output
print("Text :", text)
print("Pattern :", pattern)
print("Pattern Found At Index:", string_match(text, pattern))
```

OUTPUT



```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/benan/Downloads/DAA/Co2_AT3.py =====
Text : ABAABCABAB
Pattern : ABA
Pattern Found At Index: [0, 2, 7]
>>>
```

Explanation of the Bug

The original program uses

for i in range(n - m):

The range() function excludes the ending value.

If

- Text length = **10**
- Pattern length = **3**

then

$$n - m = 7$$

The loop executes

0,1,2,3,4,5,6

The valid starting positions are

0,1,2,3,4,5,6,7

The last possible alignment (**index 7**) is never checked.

Therefore, one possible pattern match is always skipped.

Debugging the Code

The loop should include the final valid alignment.

Incorrect

```
for i in range(n - m):
```

Correct

```
for i in range(n - m + 1):
```

The **+1** ensures that the last valid starting position is also examined.

Improvements Made

1. Fixed the loop boundary from `range(n - m)` to `range(n - m + 1)`.
 2. Used Python's `for...else` construct for clearer logic:
 - `break` exits the loop on a mismatch.
 - `else` runs only if no mismatch occurred, meaning the entire pattern matched.
 3. Added descriptive comments to improve readability and maintainability.
 4. Preserved the original brute-force behavior while making the implementation easier to understand.
-

Time Complexity Analysis

Let:

- **n** = length of the text
- **m** = length of the pattern

Best Case

The first character mismatches at every position.

Time Complexity = $O(n)$

Worst Case

The pattern matches almost completely at every position before failing (or matches fully).

The algorithm compares the pattern at every valid position.

$(n - m + 1) \times m$

Therefore,

Time Complexity = $O(n \times m)$

Space Complexity

Only a list of match indices and a few variables are used.

Space Complexity = $O(k)$

where **k** is the number of matches found (excluding the output list itself, the algorithm uses **$O(1)$** auxiliary space).

Conclusion

The original brute-force string matching implementation skipped the final valid alignment because the outer loop ended one iteration too early. By changing the loop to `range(n - m + 1)`, the algorithm checks every possible starting position and correctly identifies all occurrences of the pattern. The corrected implementation maintains the brute-force approach, improves readability with clear comments and a `for...else` structure, and has a worst-case time complexity of $O(n \times m)$.